

Project Report: Dual-Axis Solar Tracking and Energy Management System

1. Objectives

- **Maximize Energy Capture:** To dynamically track the sun's trajectory in real-time across two degrees of freedom (Yaw and Pitch), minimizing the angle of incidence to maximize the solar panel's photovoltaic efficiency.
- **Environmental Telemetry:** To integrate embedded ambient sensors to log real-time microclimate metrics (temperature and humidity) that correlate with solar output degradation.
- **Closed-Loop Power Monitoring:** To accurately log electronic metrics including open-circuit voltage, load current, and instantaneous power dissipation using high-precision I²C sensor hardware.
- **Efficient Load Regulation & Storage:** To build a safe energy-storage system utilizing dedicated battery charging profiles to prevent voltage collapse, reverse-current leakage, and thermal runaway.

2. Hardware Components List

Component Name	Technical Specifications	Purpose in Project
Arduino Uno R3	ATmega328P Microcontroller, 5V Logic	Central processing unit for sensor computation and PWM actuation.
Solar PV Panel	5V - 6V Nominal, 200mA - 500mA	Primary transducer converting solar irradiance into electrical energy.
Light Dependent Resistors (LDRs)	10kΩ Photoresistors (×4)	Arranged in a cross-axis quadrant to sense light differential.
MG996R Servo Motors	Metal Gear, High Torque, 4.8V–7.2V (×2)	Actuators for executing Horizontal (Yaw) and Vertical (Pitch) rotation.
DHT22 Sensor	Capacitive Humidity & Thermistor, Digital Data	Measures ambient environmental temperature and relative humidity.
Adafruit INA219 breakout	12-bit ADC, I ² C Interface, 0.1Ω Shunt	High-side current and bus voltage monitoring module.
TP4056 Module		

+---> (OUT +) -----+	(Pin A0) <----- Top LDR Signal
+---> (OUT -) ---> [COMMON GND]	(Pin A1) <----- Bottom LDR Signal
	(Pin A2) <----- Left LDR Signal
v	(Pin A3) <----- Right LDR Signal
[SERVOS POWER RAIL]	(Pin D2) <----- DHT22 Data Pin
(MG996R Red Wires)	(Pin D9) -----> Horiz. Servo PWM
(Yellow)	
	(Pin D10) -----> Vert. Servo PWM
(Yellow)	
	(Pin A4) <-----> INA219 SDA (Data)
	(Pin A5) <-----> INA219 SCL (Clock)

4. Full Integrated Production Code

```
#include <Servo.h>
#include <DHT.h>
#include <Wire.h>
#include <Adafruit_INA219.h>

#define DHTPIN 2
#define DHTTYPE DHT22

// Transducer Interface Pins
const int pinTop    = A0;
const int pinBottom = A1;
const int pinLeft   = A2;
const int pinRight  = A3;

// Actuator Driving Pins
const int servoH_Pin = 9;
const int servoV_Pin = 10;

// Dynamic Control Variables
const bool INVERT_VERTICAL    = false;
const bool INVERT_HORIZONTAL = false;
const int tolerance = 40; // Angular deadzone filter
const int stepSize = 1; // Intermittent motor angular step

const int limitH_Min = 10; const int limitH_Max = 170;
const int limitV_Min = 20; const int limitV_Max = 160;

DHT dht(DHTPIN, DHTTYPE);
Adafruit_INA219 ina219;
Servo servoH; Servo servoV;

int posH = 90; int posV = 90;
unsigned long prevMillis = 0;
```

```

const long telemetryInterval = 2000;

void setup() {
  Serial.begin(9600);

  servoH.attach(servoH_Pin);
  servoV.attach(servoV_Pin);
  servoH.write(posH);
  servoV.write(posV);

  dht.begin();

  if (!ina219.begin()) {
    Serial.println(F("[FATAL ERROR]: INA219 Communication Breakdown. Check I2C Interface.));
    while (1) { delay(10); }
  }

  Serial.println(F("====="));
  Serial.println(F("  2-DOF SOLAR TRACKING & SMART POWER INFRASTRUCTURE  "));
  Serial.println(F("====="));
}

void loop() {
  // Capture Light Gradients via Voltage Dividers
  int top    = 1023 - analogRead(pinTop);
  int bottom = 1023 - analogRead(pinBottom);
  int left   = 1023 - analogRead(pinLeft);
  int right  = 1023 - analogRead(pinRight);

  // Mathematical Angular Correction Logic
  int diffV = top - bottom;
  if (abs(diffV) > tolerance) {
    if (top > bottom) {
      if (INVERT_VERTICAL) posV -= stepSize; else posV += stepSize;
    } else {
      if (INVERT_VERTICAL) posV += stepSize; else posV -= stepSize;
    }
  }

  int diffH = left - right;
  if (abs(diffH) > tolerance) {
    if (left > right) {
      if (INVERT_HORIZONTAL) posH += stepSize; else posH -= stepSize;
    } else {
      if (INVERT_HORIZONTAL) posH -= stepSize; else posH += stepSize;
    }
  }

  // Enforce Boundary Conditions to Prevent Servo Stalling
  posV = constrain(posV, limitV_Min, limitV_Max);
  posH = constrain(posH, limitH_Min, limitH_Max);

  servoV.write(posV);

```

```

servoH.write(posH);

// Non-Blocking Execution of System Telemetry
unsigned long currentMillis = millis();
if (currentMillis - prevMillis >= telemetryInterval) {
    prevMillis = currentMillis;

    float humidity = dht.readHumidity();
    float temperature = dht.readTemperature();

    float busVoltage = ina219.getBusVoltage_V();
    float current_mA = ina219.getCurrent_mA();
    float power_mW = ina219.getPower_mW();

    Serial.println(F("
>>> METRIC TELEMETRY LOG <<<"));
    if (isnan(humidity) || isnan(temperature)) {
        Serial.println(F("Ambient Metrics: [Sensor Read Fault]"));
    } else {
        Serial.print(F("Ambient Metrics: Temp: ")); Serial.print(temperature, 1);
        Serial.print(F(" °C | Humidity: ")); Serial.print(humidity, 1); Serial.println(F(" %"));
    }

    Serial.print(F("Electrical Bus : Voltage: ")); Serial.print(busVoltage, 2);
    Serial.print(F(" V"));
    Serial.print(F(" | Current: ")); Serial.print(current_mA, 1); Serial.print(F(" mA"));
    Serial.print(F(" | Power: ")); Serial.print(power_mW, 1); Serial.println(F(" mW"));

    Serial.print(F("Actuator Vectors: Horizontal: ")); Serial.print(posH);
    Serial.print(F("° | Vertical: ")); Serial.print(posV); Serial.println(F("°"));
    Serial.println(F("-----"));
}

delay(30);
}

```

5. Future Scope

- **IoT Cloud Data Analytics:** Implementing an ESP32 or Wi-Fi gateway module to stream the local ambient data and electrical logs onto cloud servers (like ThingSpeak or AWS IoT) for computational analysis on solar degradation curves over time.
- **Maximum Power Point Tracking (MPPT):** Replacing the standard linear charging module with a custom microcontroller-driven buck-boost MPPT topology to maximize solar output efficiency during heavy clouds or atmospheric shading.

- **Autonomous Self-Powering Architecture:** Restructuring the energy paths such that the stored solar energy cycles through a secondary boost rail to feed power back into the microcontrollers, making the setup completely independent of external grid charging.

6. Conclusion

The designed prototype successfully presents a working framework for a dual-axis solar tracking and real-time analytical instrument. By using a quadrant-based LDR sensor module, the physical system aligns itself to the brightest optical spot with high accuracy.

The integration of isolated high-current rails using buck regulators prevents micro-controller brownouts, while the high-side INA219 instrumentation telemetry accurately quantifies load profiles without disrupting circuit parameters. The implementation of strict safe-charging protocols shields the lithium storage cells from over-voltage stress.

Ultimately, this project confirms that active dual-axis tracking significantly enhances power harvest limits compared to traditional stationary structures, presenting a viable baseline model for modern green energy systems.